

Trapezoid-Lite: A Versatile Edge Accelerator for Dense and Sparse Matrices

AOC 2026 Spring Final Project Proposal

Group 13

組長：彭俞凱 (N96141066)

組員：黃妍心 (N26141761)、王柏弘 (Q36144200)

陳秉弘 (N26140600)、楊承豫 (N26142393)

施柏安 (M16131072)

Department of Engineering Science, NCKU

May 3, 2026

Outline

- 1 Introduction & Motivation
- 2 The Trapezoid Concept
- 3 Technical Deep Dive: Zero-Skipping
- 4 Hardware Spec & Tiling Calculation
- 5 Analytical Evaluation
- 6 Implementation & Organization

1.1 Background: The Sparsity Spectrum in AI

矩陣運算中稀疏度 (Sparsity) 的多樣性探討

- 在當前深度學習與圖形分析領域，神經網路矩陣的稀疏度呈現極大的跨度。
- **Dense 至 Mildly Sparse (輕度稀疏)**: 如未經壓縮的卷積層，或是經剪枝 (Pruning) 後留下 10%~30% 非零元素的權重矩陣。
- **Highly Sparse (高度稀疏)**: 如圖神經網路 (GNN) 或科學計算，其稀疏度往往高達 99% 以上。
- **硬體設計之挑戰**: 單一加速器難以在跨度如此巨大的稀疏矩陣中皆保持高效能。

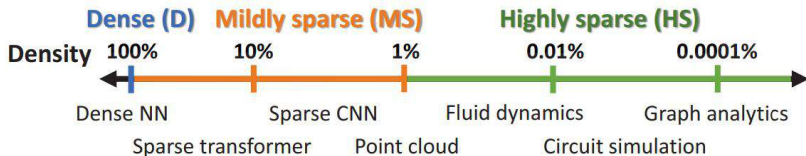


Figure 1: Application domains across the matrix sparsity spectrum.

1.2 Limitations of Conventional Accelerators

現行架構在處理混合稀疏矩陣時的效能瓶頸

- Systolic Array 架構 (如 Google TPU):

- 優勢在於能極高效地處理 Dense $\times$ Dense 矩陣運算。
- 缺陷: 缺乏跳過零值的機制。當處理經由 ReLU 或剪枝產生的稀疏矩陣時，系統會耗費大量運算週期執行無效的 $0 \times X = 0$ 操作，導致硬體利用率 (Compute Utilization) 顯著下降。

- Sparse-only 架構 (如 Flexagon, SIGMA):

- 為高度稀疏矩陣設計，具備複雜的交集與路由硬體。
- 缺陷: 處理 Dense 矩陣時，過於龐大的索引負載 (Indexing Overhead) 與有限的乘法器數量，會大幅限制系統的峰值吞吐量 (Peak Throughput)。

2.1 The Trapezoid Solution: A Versatile Approach

Trapezoid 加速器之核心理念與預期效能

- 本專題參考 Trapezoid 論文之架構，旨在設計一款能「動態適應」各級稀疏度的通用邊緣加速器 (Versatile Edge Accelerator)。
- **全頻段適應性:** 如下圖效能熱力圖所示，Trapezoid 藉由硬體重構，在 $D \times D$, $MS \times D$, 甚至 $HS \times HS$ 等各種情境下，均能維持優異的運算效能，克服了傳統加速器的偏科問題。
- 4 dataflow 應對不同情況: Standard IP, TrIP, TrGS, and TrGT



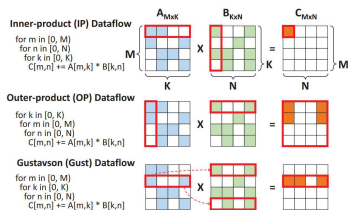
Figure 2: Performance comparison across diverse sparsity combinations.

2.2 Top-Level Architecture Overview

硬體架構：結合 **2D PE 陣列** 與 **動態稀疏處理單元**

● 系統基於傳統的 2D 乘加陣列進行擴充，包含以下關鍵模組：

- 1 **Multi-Fiber Intersection Unit (MFIU)**: 接收輸入矩陣之位元遮罩，進行硬體層級之邏輯交集，定位有效運算。
- 2 **Distribution Networks**: 依據 MFIU 之運算結果，將非零資料精確繞線至閒置之 Processing Element (PE)。
- 3 **Merge-Reduction Tree**: 負責將空間映射 (Spatial Mapping) 產生之多個 Partial Sums 進行平行加總。



3.1 Technical Detail: Bitmask Representation

資料壓縮與表示：位元遮罩 (Bitmask) 機制

- 爲了降低記憶體頻寬消耗並加速零值判斷，系統採用 Bitmask 格式來表示稀疏向量 (Fiber)。
- 編碼原理：
 - 向量中的每一個元素對應 Bitmask 中的一個位元 (Bit)。
 - 若元素爲非零數值，該位元設爲 1；若爲零，則設爲 0。
 - 實際的非零數值 (Non-zero values) 則緊湊地連續儲存於記憶體中。
- 效益：硬體可先讀取極小容量的 Bitmask 進行邏輯判斷，確認有交集後，才從記憶體中提取真正的數值，大幅節省功耗。

3.2 Technical Detail: MFIU Operation

有效運算之篩選：MFIU 交集與前綴和電路

- **Intersection (邏輯交集):** 將矩陣 A (Row) 與矩陣 B (Column) 的 Bitmask 送入硬體 AND 邏輯閘陣列。輸出結果中的 1 即代表該座標上兩者皆有數值，需進行乘法運算。
- **Prefix Sum (前綴和運算):**
 - 交集結果隨即送入硬體前綴和電路 (Prefix Sum Tree)。
 - 該電路會計算每個有效運算的「絕對索引值」。此索引值等同於將該運算動態發配給 PE 陣列中的**第 N 個乘法器**。

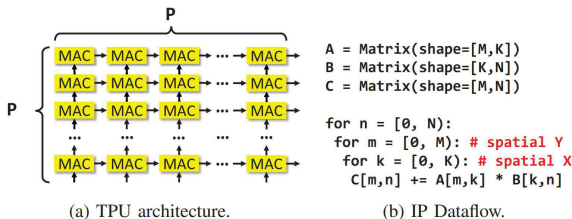


Figure 3: Microarchitecture of the Multi-Fiber Intersection Unit (MFIU).

4.1 Dataflow Optimization: TrIP (Inner-product-based)

針對輕度稀疏矩陣之資料流排程 (TrIP Dataflow)

- 在面對 Mildly Sparse 矩陣時，若僅採用單行單列之內積，會因有效交集過少導致硬體利用率低落。
- Fiber Packing (多維度打包機制):
 - TrIP 資料流在單一週期內，會同時從 SRAM 中抓取多條 **Rows** (矩陣 A) 與多條 **Columns** (矩陣 B)。
 - 透過 MFIU 同時對這多組向量進行交集分析。即使單組向量的交集率低，累積起來的有效運算量仍足以填滿 **PE Row** 內的所有乘法器。

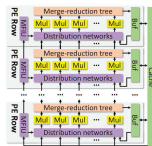


Figure 4: TrIP packs multiple sparse fibers to maximize multiplier utilization.

4.2 Hardware Spec & ADFP Constraints

考量 ADFP 實務限制之實體架構縮放 (Hardware Scaling)

- **ADFP 實體限制:** ADFP 製程並不包含彈性的 Memory Compiler。經查閱規範，最大之 Two-port SRAM Hard Macro 容量僅為 **1 KB** ($128 \text{ words} \times 64 \text{ bits}$)。若套用原論文 17 MB 規格，需實例化高達 1.7 萬顆 Macro，將導致 P&R 繞線全面癱瘓。
- **Proposed "Lite" Spec (精確縮放後規格):**
 - ❶ **PE Array [16×16]:** 縮減為 256 MACs。由於 Trapezoid 的動態路由網路 (Distribution Network) 繞線複雜度極高， 16×16 既保留了 2D 陣列的資料重用 (Data Reuse) 優勢，又能避免控制邏輯佔比過高導致時序 (Timing) 無法收斂。
 - ❷ **Global Buffer [16 KB]:** 藉由實例化 16 顆 **1 KB (128×64) Two-port SRAM** 拼接而成。此數量在平面規劃 (Floorplan) 具備對稱性，確保了晶片佈局與繞線之高度可行性。

4.4 hardware simulation

32 KB SRAM 是否足夠？基於 **Tiling** 之工作集精算

- 假設系統採 INT8 量化，配合 16×16 PEs，系統每次自 DRAM 載入 32×32 大小之矩陣切塊 (Tile) 進行運算：
 - 矩陣 **A Tile (INT8)**: $32 \times 32 \times 1 \text{ Byte} = 1 \text{ KB}$
 - 矩陣 **B Tile (INT8)**: $32 \times 32 \times 1 \text{ Byte} = 1 \text{ KB}$
 - 矩陣 **C Tile (Partial Sums)**: 為防精度溢位採 INT32 $\rightarrow 32 \times 32 \times 4 \text{ Bytes} = 4 \text{ KB}$
 - **MFIU** 位元遮罩 (Bitmasks): A 與 B 共 2048 元素 $\rightarrow 2048 \text{ bits}/8 = 0.25 \text{ KB}$
- **Working Set (單次運算總量)**: $1 + 1 + 4 + 0.25 = 6.25 \text{ KB}$
- **Double Buffering (乒乓緩衝設計)**: 為隱藏 DRAM 存取延遲，使 PE 不中斷運算，需準備雙倍容量進行切換 $\rightarrow 6.25 \text{ KB} \times 2 = 12.5 \text{ KB}$
- **結論**: 16 KB (16 Banks) 之配置能完美且游刃有餘地容納 12.5 KB 之核心工作集需求，證明本硬體規格設計具備嚴謹之合理性。

5.1 Analytical Report: Compute & Bandwidth Estimates

硬體效能理論值估算 (Lab 2 Analytical Model Extension)

- 系統目標時脈頻率設定為 **500 MHz**。
- Peak Performance (峰值算力):

$$16 \times 16 \text{ PEs} \times 2 \text{ (MAC Ops)} \times 0.5 \text{ GHz} = \mathbf{256 \text{ GOPS}}$$

- Peak Bandwidth (峰值內部頻寬):
 - 假設 16 Banks SRAM 每個SRAM 1 word(64bit)/cycle
 - 最高總匯流排寬度 128 Bytes/cycle: $128 \text{ Bytes} \times 0.5 \text{ GHz} = \mathbf{64 \text{ GB/s}}$
- Arithmetic Intensity (算術強度):

$$\text{Ridge Point} = 256 \text{ GOPS} / 128 \text{ GB/s} = \mathbf{2 \text{ OPs/Byte}}$$

- 考慮到記憶體使用效率與藉由 32×32 之 Tiling 策略帶來的資料重用，系統仍很有機會滿足 $3 \sim 5 \text{ OPs/Byte}$ 之強度門檻，確保多數運算落於高效之 Compute-bound 區間。

5.2 Roofline Model Analysis

系統效能邊界分析 (Roofline Model)

- 傳統加速器在處理稀疏矩陣時，因需要讀取大量無效的 0 值，導致實際的 Arithmetic Intensity 驟降，落入 Memory-bound 區域。
- Trapezoid 預期優勢: 藉由 Bitmask 壓縮與 TriP 資料流的高資料重用率 (Data Reuse)，有效提升 OPs/Byte。理論上能使運算效能緊貼 Roofline 邊界，最大化硬體吞吐量。

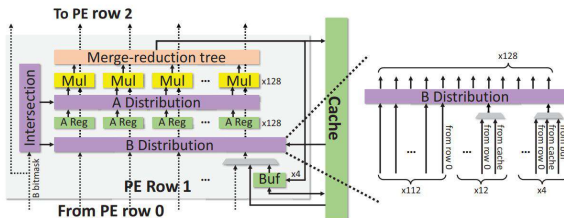


Figure 5: Expected Roofline model for the proposed accelerator architecture.

6.1 End-to-End System Integration Flow

軟硬協同設計與系統驗證流程

- 本專題將實作完整的 End-to-End Inference 流程：
- Software Golden Model (Python):
 - 開發資料預處理腳本，對輸入模型之權重進行量化 (Quantization)。
 - 執行矩陣稀疏化處理，並生成對應之 **Bitmask** 檔案與測試資料。
- Hardware RTL (Verilog):
 - 以暫存器傳遞層級 (RTL) 撰寫 16×16 PE Array、MFIU 與 Memory Controller。
- Co-simulation (Verilator + C++):
 - 於 Ubuntu 24.04 環境中，利用 Verilator 將硬體編譯為 C++ 模型。
 - 結合 Python Wrapper 進行跨語言介接，自動化驗證硬體輸出之正確性。
- verify performance:
 - 在RTL中增加counter來比較TrIP vs TPU(standard mode)實際需要的cycle。
 - 使用Design Compiler來驗證TrIP dataflow是否能執行500MHZ。

6.2 Division of Labor (All Coding Team)

團隊分工規劃與協作規範

- 團隊 6 名成員皆配置程式開發任務，並統一經由 GitHub 進行版本控制與審查 (Pull Request)。

Member	Key Responsibilities (Development & Verification)
彭俞凱 (Leader)	MFIU (Bitmask Intersection & Prefix Sum) RTL 邏輯開發
黃妍心	16 × 16 PE Array 內部乘加單元與 Pipeline 管線設計
施柏安	Merge-Reduction Tree & Distribution Network 繞線設計
陳秉弘	Global Buffer (ADFP SRAM Banks) 與 Memory Controller
楊承豫	Python Golden Model (Quantization, Bitmask Generation)
王柏弘	Verilator 系統整合 (C++ Wrapper) 與 Testbench 自動化

6.3 Risk Management & Fallback Plan

風險管理與彈性實作

- 潛在風險分析:

- Trapezoid 之 TrIP 資料流需進行多條向量之平行路由，其 Distribution Network 之 Verilog 實作複雜度極高，可能面臨時序 (Timing) 無法收斂或繞線衝突問題。

- 退場降級機制:

- 若於專案第 5 週 (Week 5) MFIU 模組仍無法順利與 PE 陣列整合，將啟動降級方案。
- 方案 A: 退回支援單邊稀疏 (1D Sparsity) 之架構，簡化路由邏輯。
- 方案 B: 關閉 MFIU 功能，還原為純 Dense 運算之 Baseline Systolic Array。
- 藉此確保期末評分階段，必有具備完整 End-to-End Inference 功能之系統可供驗證展示。

- 進一步實作:TrGT

- 實現Temporal Gustavson Dataflow
- 找SuiteSparse裡面的ca-CondMat/GNN model測試 $HS \times HS$

- 彈性調整memory

- 若記憶體空間不足以最大化運算效率將考慮將SRAM從16改成32並適當改變運作頻率
- 若memory routing部分出問題可以把SRAM改成single port並修改硬體架構

6.4 Project Timeline

專案時程表 (Gantt Chart)

Task / Timeline	W1	W2	W3	W4	W5	W6	W7	W8
Software Golden Model	■	■						
RTL: MFIU & PE Logic		■	■	■				
RTL: SRAM Ctrl & NoC			■	■	■			
Verilator Integration					■	■		
System Debugging (Buffer)						■	■	
Analysis & Evaluation							■	
Report & Slides (6/21)								■

- 時程管理策略: 強制將硬體開發與初步模組整合於前 5 週完成。保留第 6 至第 7 週作為 **System Debugging** 緩衝期，專注於解決 Verilator 整合時潛在之資料流錯誤。

Thank you for your time and attention